



Image Style Transfer

Sophie Juco, Joy Fan, Simone Rittenhouse



Style Transfer Overview

01

What is Style Transfer?

Content
Image



Style Image



Generated Image
(Gatys et al., 2016)

Image Style Transfer Using Convolutional Neural Networks (Gatys et al., 2016)

Generate an image that combines the content of one image with the style of another using a Convolutional Neural Network:

- Content is captured from deeper blocks of a pretrained VGG-19 network, focuses less on small textures and more on big-picture structure.

- Style is captured from shallow to deeper blocks of a pretrained VGG-19 network, focusing on textures, colors, and patterns rather than overall structure.

- A loss function combines content and style losses. The output image optimized iteratively to minimize this loss.

$$\text{Total Loss} = \text{CW}(\text{content loss}) + \text{SW}(\text{style loss})$$

Our Style Transfer Implementation

Our implementation follows the Gatys et al. paper with a few differences:

Uses Adam optimizer (instead of L-BFGS) for updating the generated image.

Uses max pooling in the VGG network instead of average pooling and does not normalize the VGG weights.

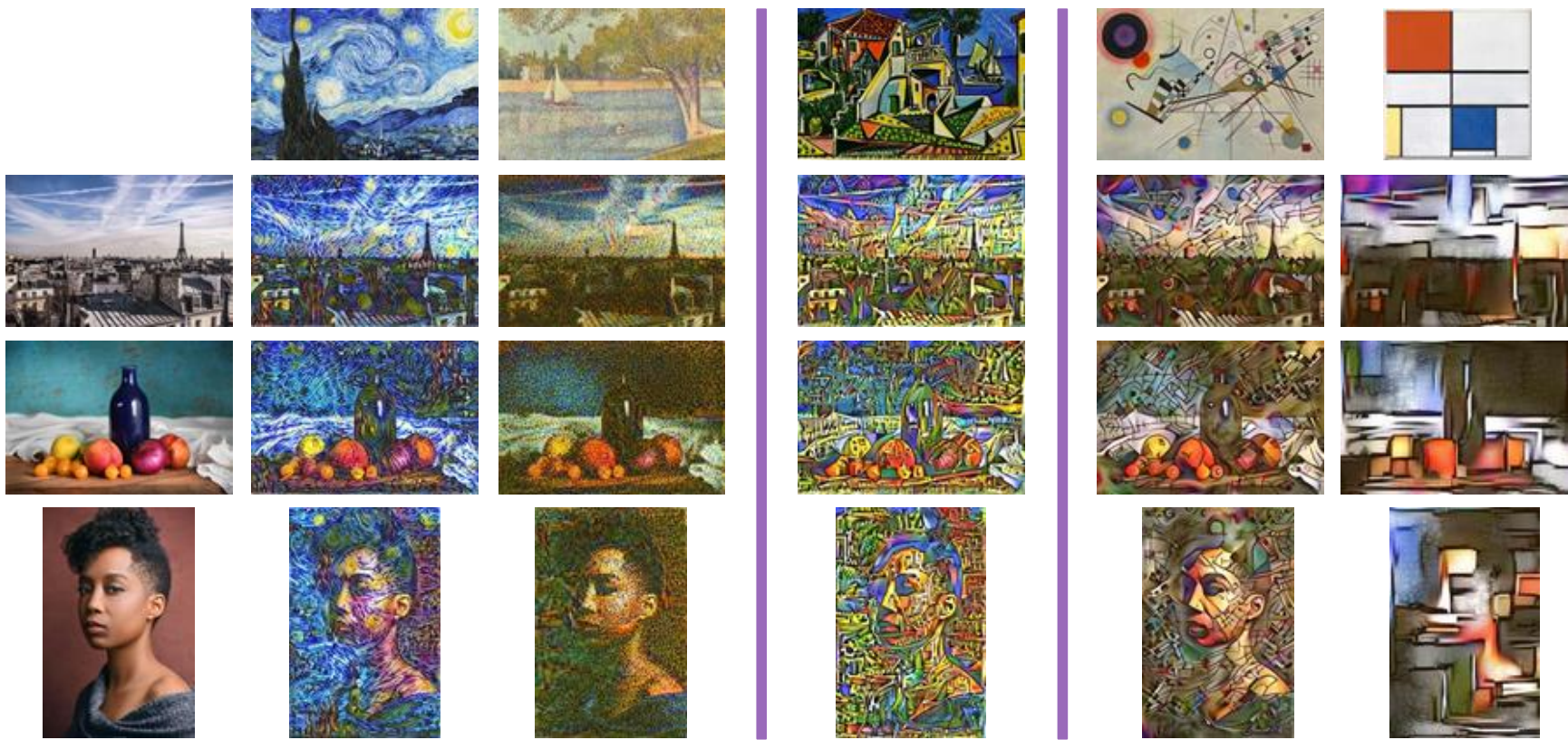
In addition to applying a single style image, we also blend two different style images using a parameter α to control the weight.

Style Transfer Results

02

Impressionism VS Abstract Style

*CW:1, SW: 1e6



Impressionism

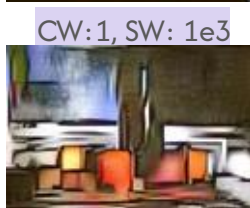
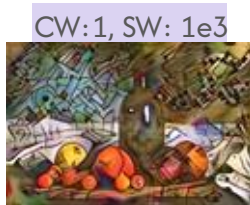
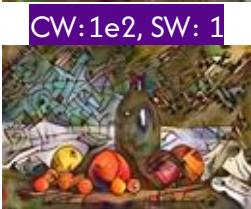
Cubism

Abstract

Style VS Content Weight - Landscape



Style VS Content Weight - Still Life



Style VS Content Weight - Portrait



CW:1, SW: 1



CW:1, SW: 1e1



CW:1, SW: 1e2



CW:1, SW: 1e3



CW:1, SW: 1



CW:1, SW: 1e1



CW:1, SW: 1e2



CW:1, SW: 1e3



CW:1e2, SW: 1



CW:1e3, SW: 1



CW:1e4, SW: 1



CW:1e6, SW: 1



CW:1e2, SW: 1



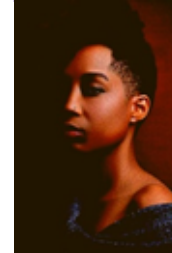
CW:1e3, SW: 1



CW:1e4, SW: 1



CW:1e6, SW: 1



Applying two different styles at the same time

25% style 1
75% style 2



50% style 1
50% style 2



75% style 1
25% style 2



Style 1



Style 2



Applying two different styles at the same time

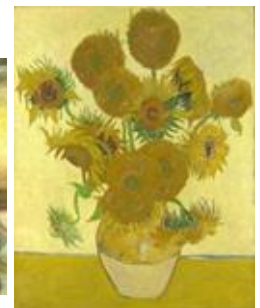
25% style 1
75% style 2

50% style 1
50% style 2

75% style 1
25% style 2

Style 1

Style 2



Denoising Overview

03

Denoising Overview

Built-in OpenCV Functions

- `cv2.GaussianBlur(src, ksize, sigmaX):`
Applies a Gaussian kernel to blur the image
- `cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace):`
Smooths the image while keeping edges sharp by considering both pixel distance and color similarity

Non-Local Means (Buades et al., 2005)

- Denoised pixel ($x[i]$) is weighted sum of all image pixels ($y[j]$) in a search window S
weights are between patches y_i and y_j

$$x[i] = \sum_{j \in S} \frac{K_h(y_i - y_j)}{\sum_{j \in S} K_h(y_i - y_j)} y[j]$$

- Buades et al. used grayscale images:
We convert RGB images to $L^*a^*b^*$ and run NLM separately on the channels

Denoising Results

04

Non-Local Means Denoising



Style Transfer Result

CW = $1e3$
SW = 1



NLM Result

$h = 20$,
patch size = 7,
search size = 21



Built-in OpenCV Functions

Light



Medium



Strong



Gaussian

- Blurs both textures and edges uniformly.
- Image looks soft and less detailed overall.



Bilateral

- Preserves edges better than Gaussian while still reducing noise.

Thank you!

References:

1. Gatys, L. A., Ecker, A. S., & Bethge, M. (2016). Image style transfer using Convolutional Neural Networks. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2414–2423.
<https://doi.org/10.1109/cvpr.2016.265>
2. Buades, A., Coll, B., & Morel, J.-M. (2005). A non-local algorithm for image denoising. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2, 60–65. <https://doi.org/10.1109/cvpr.2005.38>